Git Interview Questions

🔀 Rebase vs Merge

Merge

- Combines branches with a *merge commit*
- Preserves full history (including branch structure)
- Safe for shared branches

Rebase

- Rewrites history by placing your commits on top of another branch
- Creates a cleaner, linear history
- Dangerous on shared branches (can break others' work)

👉 **Rule of thumb:**
Use **merge** for collaboration, **rebase** for local cleanup.

🚑 Hotfix

- A quick fix for a critical issue in production
- Typically branched from `main` (or `master`)
- After fixing, merged back into both:
  - `main`
  - `develop` (if using Git Flow)

PATHNEX
Shaping Careers, Building Futures

🍒 Cherry-pick

- Apply a specific commit from one branch to another

```
git cherry-pick <commit-hash>
```

👉 Useful when:

- You need one fix but not the whole branch

🌳 Branching Strategies

1. Git Flow

- `main` → production
- `develop` → ongoing work
- feature / release / hotfix branches

2. GitHub Flow

- Simple: `main` + feature branches + PRs

3. Trunk-Based Development

- Everyone commits to `main` frequently
- Short-lived branches

👉 Choose based on team size & release complexity.

📄 .gitignore

- Tells Git what files to ignore

Example:

```
node_modules/
.env
*.log
```

👉 Prevents committing:

- Secrets
- Dependencies
- Build files

⚔️ How to Resolve Git Conflict

1. Conflict occurs during merge/rebase
2. Open conflicting file:

```
<<<<<<< HEAD
your code
=======
incoming code
>>>>>>> branch
```

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

3. Edit manually → keep correct version
4. Then:

```
git add .
git commit
```

👉 For rebase:

```
git rebase --continue
```

🔄 Hard vs Soft Reset

Soft Reset

```
git reset --soft HEAD~1
```

- Undo commit
- Keeps changes staged

Hard Reset

```
git reset --hard HEAD~1
```

- Deletes commit + changes permanently

👉 Use hard reset carefully (destructive)

📦 Git Stash

- Temporarily save uncommitted changes

```
git stash
git stash pop
```

👉 Useful when switching branches mid-work

🍴 Git Clone vs Fork

Clone

```
git clone <repo-url>
```

- Copy repo to your local machine

PATHNEX
Shaping Careers, Building Futures

Fork

- Copy repo to your GitHub account
- Used for contributing to others' projects

👉 Fork → for collaboration
👉 Clone → for local development

🔽 Pull vs Fetch

Fetch

```
git fetch
```

- Downloads changes
- Doesn't modify your working branch

Pull

```
git pull
```

- Fetch + merge

👉 Safer workflow:

```
git fetch
git merge
```

PATHNEX
Shaping Careers, Building Futures

🔁 Reset vs Revert

Reset

- Moves branch pointer (rewrites history)

Revert

```
git revert <commit>
```

- Creates a new commit that undoes changes
- Safe for shared branches

👉 Use:

- **reset** → local changes
- **revert** → shared history

🧠 Quick Summary

Concept	Safe for Team?	Rewrites History?
Merge	✅ Yes	❌ No
Rebase	⚠️ No (shared)	✅ Yes
Reset	⚠️ No	✅ Yes
Revert	✅ Yes	❌ No

PATHNEX
Shaping Careers, Building Futures